



Lab02: Querying Database Tables

Objectives:

To learn the SQL WILDCARDS and ORDER BY clauses in SELECT statement.

1. SQL WILDCARDS

In SQL, wildcards are special characters that are used in conjunction with the **LIKE** operator to perform [pattern-matching](#) searches within string data.

SQL wildcards are used to search for data within a table.

TYPES OF SQL WILDCARDS

01

Percent Sign (%) Wildcard

02

Underscore (_) Wildcard

03

Bracket ([]) Wildcard

04

Caret (^) Wildcard

For Microsoft SQL Server, the wildcard characters are defined as follows:

Wildcard	Description
_ (underscore)	matches any single character
%	matches a string of one or more characters
[]	matches any single character within the specified range (e.g. [a-f]) or set (e.g. [abcdef]).
[^]	matches any single character not within the specified range (e.g. [^a-f]) or set (e.g. [^abcdef]).

A few examples should help clarify these rules.

- **WHERE** FirstName **LIKE** '_im' finds all three-letter first names that end with 'im' (e.g. Jim, Tim).
- **WHERE** LastName **LIKE** '%stein' finds all employees whose last name ends with 'stein'
- **WHERE** LastName **LIKE** '%stein%' finds all employees whose last name includes 'stein' anywhere in the name.
- **WHERE** FirstName **LIKE** '[JT]im' finds three-letter first names that end with 'im' and begin with either 'J' or 'T' (that is, *only* Jim and Tim)
- **WHERE** LastName **LIKE** 'm[^c]%' finds all last names beginning with 'm' where the following (second) letter is not 'c'.

Here too, we can opt to use the NOT operator: to find all of the employees whose first name does not start with 'M' or 'A', we would write

```
SELECT EmployeeID, FirstName, LastName, HireDate, City FROM Employees
```

```
WHERE (FirstName NOT LIKE 'M%') AND (FirstName NOT LIKE 'A%')
```

resulting in

	EmployeeID	FirstName	LastName	HireDate	City
	1	Nancy	Davolio	1/5/1992 12:00:00 AM	Seattle
	3	Janet	Leverling	1/4/1992 12:00:00 AM	Kirkland
	5	Steven	Buchanan	17/10/1993 12:00:00 AM	London
	7	Robert	King	2/1/1994 12:00:00 AM	London
	8	Laura	Callahan	5/3/1994 12:00:00 AM	Seattle

2. ORDER BY Clause

Until now, we have been discussing filtering the data: that is, defining the conditions that determine which rows will be included in the final set of rows to be fetched and returned from the database.

Once we have determined which columns and rows will be included in the results of our **SELECT** query, we may want to control the order in which the rows appear—sorting the data.

To sort the data rows, we include the **ORDER BY** clause. The **ORDER BY** clause includes one or more column names that specify the sort order. If we return to one of our first **SELECT** statements, we can sort its results by City with the following statement:

```
SELECT EmployeeID, FirstName, LastName, HireDate, City FROM Employees
ORDER BY City
```

By default, the sort order for a column is ascending (from lowest value to highest value), as shown below for the previous query:

	EmployeeID	FirstName	LastName	HireDate	City
	3	Janet	Leverling	1/4/1992 12:00:00 AM	Kirkland
	5	Steven	Buchanan	17/10/1993 12:00:00 AM	London
	6	Michael	Suyama	17/10/1993 12:00:00 AM	London
	7	Robert	King	2/1/1994 12:00:00 AM	London
	9	Anne	Dodsworth	15/11/1994 12:00:00 AM	London
	4	Margaret	Peacock	3/5/1993 12:00:00 AM	Redmond
	1	Nancy	Davolio	1/5/1992 12:00:00 AM	Seattle
	8	Laura	Callahan	5/3/1994 12:00:00 AM	Seattle
	2	Andrew	Fuller	14/8/1992 12:00:00 AM	Tacoma

If we want the sort order for a column to be descending, we can include the **DESC** keyword after the column name.

The **ORDER BY** clause is not limited to a single column. You can include a comma-delimited list of columns to sort by—the rows will all be sorted by the first column specified and then by the

next column specified. If we add the Country field to the **SELECT** clause and want to sort by Country and City, we would write:

```
SELECT EmployeeID, FirstName, LastName, HireDate, Country, City FROM
Employees ORDER BY Country, City DESC
```

Note that to make it interesting, we have specified the sort order for the City column to be descending (from highest to lowest value). The sort order for the Country column is still ascending. We could be more explicit about this by writing

```
SELECT EmployeeID, FirstName, LastName, HireDate, Country, City FROM
Employees ORDER BY Country ASC, City DESC
```

But this is not necessary and is rarely done. The results returned by this query are

EmployeeID	FirstName	LastName	HireDate	Country	City
5	Steven	Buchanan	17/10/1993 12:00:00 AM	UK	London
6	Michael	Suyama	17/10/1993 12:00:00 AM	UK	London
7	Robert	King	2/1/1994 12:00:00 AM	UK	London
9	Anne	Dodsworth	15/11/1994 12:00:00 AM	UK	London
2	Andrew	Fuller	14/8/1992 12:00:00 AM	USA	Tacoma
1	Nancy	Davolio	1/5/1992 12:00:00 AM	USA	Seattle
8	Laura	Callahan	5/3/1994 12:00:00 AM	USA	Seattle
4	Margaret	Peacock	3/5/1993 12:00:00 AM	USA	Redmond
3	Janet	Leverling	1/4/1992 12:00:00 AM	USA	Kirkland

It is important to note that a column does not need to be included in the list of selected (returned) columns in order to be used in the **ORDER BY** clause. If we don't need to see/use the Country values, but are only interested in them as the primary sorting field we could write the query as



```
SELECT EmployeeID, FirstName, LastName, HireDate, City FROM Employees  
ORDER BY Country ASC, City DESC
```

With the results being sorted in the same order as before:

	EmployeeID	FirstName	LastName	HireDate	City
	5	Steven	Buchanan	17/10/1993 12:00:00 AM	London
	6	Michael	Suyama	17/10/1993 12:00:00 AM	London
	7	Robert	King	2/1/1994 12:00:00 AM	London
	9	Anne	Dodsworth	15/11/1994 12:00:00 AM	London
	2	Andrew	Fuller	14/8/1992 12:00:00 AM	Tacoma
	1	Nancy	Davolio	1/5/1992 12:00:00 AM	Seattle
	8	Laura	Callahan	5/3/1994 12:00:00 AM	Seattle
	4	Margaret	Peacock	3/5/1993 12:00:00 AM	Redmond
	3	Janet	Leverling	1/4/1992 12:00:00 AM	Kirkland